



Understanding Middleware with Apache ActiveMQ

Even though 'middleware' might seem like a close cousin of Tolkien's Middle Earth, it's a fairly modern term in computer software and services, and has nothing to do with hobbits, wizards or adventures involving a ring. Middleware is nothing but software that literally sits 'in the middle', like a bridge, between other software. Read on as I shed light on one specific example of middleware, Apache ActiveMQ.

Before we explore Apache's middleware offering, it's important to know the various types of middleware out there. Middleware are classified into types based on the primary functions they perform. A popular classification is by Judith Hurwitz, in her paper, 'Sorting Out Middleware', according to which middleware can be:

- Embedded middleware: As the name suggests, this type of middleware handles embedded applications (software or firmware).
- RPC middleware: RPC (Remote Procedure Call) middleware communicates with applications using calls
- Object request broker: Here, applications can send and receive objects.
- SQL-oriented data access middleware: Primarily meant for database-related applications, this class

of middleware sits between applications that communicate with databases.

- Message-oriented middleware: This is where Apache ActiveMQ fits in. Message-oriented middleware mediate the communication or messages, between applications.

Introducing Apache ActiveMQ

One of the best open source solutions for message broking is Apache ActiveMQ. It is a prime product used in enterprise environments, because it supports advanced features such as multiple instances to store messages, and clustering environments. The backbone of ActiveMQ is what's known as JMS—the Java Messaging Service. JMS is an API implementation within J2EE (Java Enterprise) which is specifically for message-oriented applications.